

A.L.I.C.E.

Assistenza per la Longevità Intelligente, Connessa, Empatica

Documentazione di Progetto Piattaforma Web per il Monitoraggio Remoto di Pazienti Anziani

Anno Accademico:

2025/2026

Esame:

PROGRAMMAZIONE PER IL WEB

Realizzato da

Manuel	Manna	803080	m.manna6@studenti.uniba.it
Alessio	Mininni	798191	a.mininni14@studenti.uniba.it
Andrius	Tamuliunaite	800256	a.tamuliunaite@studenti.uniba.it

Indice

1	Descrizione Informale del Contesto e degli Obiettivi	3
1.1	Obiettivi generali	3
1.2	Funzionalità implementate	3
2	Individuazione degli Stakeholder e dei Goal	4
2.1	Stakeholder e Goal	4
2.1.1	Operatore Sanitario / Caregiver	4
2.1.2	Paziente Anziano	4
2.2	Diagramma di Contesto	5
2.3	Scambio di Valore	5
3	Stato dell'Arte, Analisi della Concorrenza e Analisi "Make or Buy"	6
3.1	Stato dell'Arte	6
3.2	Analisi della Concorrenza	6
3.2.1	MyTherapy	6
3.2.2	ADiLife	6
3.3	Analisi "Make or Buy"	7
4	Modello Informativo	8
4.1	Diagramma EER	8
4.2	Modello Relazionale	10
4.3	Stima Dimensionale del Database	11
4.3.1	Stima per entità	11
4.3.2	Tabella di popolamento e crescita	12
4.4	Modello di Visibilità	12
5	Modello di Navigazione	14
5.1	Legenda	14
5.2	Albero di navigazione reale	14
5.3	Descrizione delle viste	14
5.3.1	Area comune	14
5.3.2	Area operatore	15
5.3.3	Area paziente	15
5.4	Regole di protezione e reindirizzamento	15
5.5	Principi di navigazione	15

6	Modello Architettuale	17
6.1	Architettura hardware	17
6.2	Modello architettuale	17
6.3	Architettura software	18
6.4	Responsabilita' dei livelli	18
6.5	Stack tecnologico	19
6.6	DBMS, browser e sistemi supportati	19
7	Aspetti Implementativi	20
7.1	Struttura del progetto	20
7.2	Pagine e route	20
7.3	Server Actions e accesso ai dati	20
7.4	Componenti client	21
7.5	Helper di supporto	21
7.6	Suggerimento AI	21
7.7	Sicurezza	21
8	Test	22
8.1	Strumenti utilizzati	22
8.2	Casi di test principali	22
8.3	Verifiche sul database	22
9	Conclusioni	24
9.1	Risultati raggiunti	24
9.2	Benefici attesi	24
9.3	Limiti attuali	24
9.4	Sviluppi futuri	25
10	Matrice RACI	26
10.1	Ruoli del team	26
10.2	RACI della documentazione	26
10.3	RACI dello sviluppo	27

1 Descrizione Informale del Contesto e degli Obiettivi

Il progetto A.L.I.C.E. nasce dall'esigenza di supportare il monitoraggio quotidiano di pazienti anziani fragili, soprattutto quando l'assistenza non avviene sempre in presenza. In molti contesti familiari e sanitari il paziente segue una terapia, svolge piccoli esercizi fisici e comunica il proprio stato d'animo in modo frammentato: una parte viene riferita verbalmente, una parte viene annotata dall'operatore e una parte rischia di non essere registrata. Questo rende piu' difficile avere una visione continua dell'andamento del paziente.

L'applicazione propone una piattaforma web con due esperienze distinte. Da un lato c'e' l'operatore sanitario, che ha bisogno di una dashboard ordinata per controllare piu' pazienti, verificare l'aderenza ai farmaci, controllare gli esercizi completati e leggere lo storico dell'umore. Dall'altro lato c'e' il paziente anziano, che non deve usare un gestionale complesso, ma solo compiere poche azioni semplici: indicare come si sente, confermare l'assunzione dei farmaci e completare gli esercizi assegnati.

La scelta di una web application consente di evitare installazioni dedicate e permette l'accesso da dispositivi comuni, come PC e tablet. L'interfaccia del paziente e' pensata per ridurre il carico cognitivo: le pagine sono brevi, i percorsi sono lineari e le azioni principali sono presentate tramite card e pulsanti ben visibili. L'interfaccia dell'operatore, invece, e' piu' informativa e raccoglie dati, metriche e strumenti di gestione in un'unica area.

Il sistema non vuole sostituire il giudizio clinico dell'operatore. Il suo obiettivo e' fornire un supporto organizzativo e informativo, raccogliendo in modo strutturato dati che altrimenti sarebbero dispersi. Per questo motivo A.L.I.C.E. integra anche un suggerimento AI rivolto esclusivamente all'operatore: il suggerimento non formula diagnosi, ma riassume una possibile micro-azione operativa a partire da dati sintetici come aderenza ai farmaci, completamento degli esercizi e ultimo umore registrato.

1.1 Obiettivi generali

Gli obiettivi principali del progetto sono quattro. Il primo e' **semplificare l'interazione del paziente**, rendendo l'uso quotidiano dell'applicazione comprensibile anche a utenti non abituati a strumenti digitali complessi. Il secondo e' **centralizzare le informazioni utili all'operatore**, evitando che farmaci, esercizi e umore siano controllati con strumenti separati. Il terzo e' **misurare l'aderenza giornaliera**, cosi' da individuare rapidamente eventuali cali nella routine del paziente. Il quarto e' **mantenere il progetto tecnicamente semplice**, usando Next.js, React, Supabase e poche astrazioni, in modo che il codice resti spiegabile e manutenibile.

1.2 Funzionalita' implementate

Nella versione attuale sono implementati la registrazione e il login separati per operatore e paziente, la dashboard dell'operatore, la pagina di dettaglio paziente, la gestione di farmaci ed esercizi, la registrazione dell'umore, la conferma giornaliera dei farmaci, il completamento guidato degli esercizi e il suggerimento AI per l'operatore. Tutte queste funzionalita' sono collegate allo schema Supabase del progetto e rispettano la separazione tra area operatore e area paziente.

2 Individuazione degli Stakeholder e dei Goal

2.1 Stakeholder e Goal

Gli stakeholder principali del sistema sono l'**Operatore Sanitario** e il **Paziente Anziano**. I due attori usano la stessa piattaforma, ma con obiettivi e livelli di complessita' diversi. L'operatore deve gestire e interpretare i dati; il paziente deve svolgere azioni quotidiane semplici.

2.1.1 Operatore Sanitario / Caregiver

L'operatore sanitario e' il professionista che segue uno o piu' pazienti. Attraverso la dashboard puo' consultare l'elenco dei pazienti assegnati, entrare nel dettaglio di ciascun paziente, aggiornare alcune informazioni, configurare farmaci ed esercizi e monitorare i log giornalieri.

Stakeholder	Goal	Subgoal	C	R	U	D
Operatore Sanitario	Gestione pazienti	Visualizza lista pazienti		X		
		Modifica dati del paziente			X	
		Consulta dettaglio e metriche		X		
Operatore Sanitario	Gestione farmaci	Aggiunge farmaco	X			
		Visualizza farmaci assegnati		X		
		Elimina farmaco				X
Operatore Sanitario	Gestione esercizi	Crea esercizio con step	X			
		Visualizza esercizi assegnati		X		
		Elimina esercizio				X
Operatore Sanitario	Monitoraggio	Consulta umore, farmaci ed esercizi		X		
		Richiede suggerimento AI	X			

Tabella 1: Goal e operazioni CRUD dell'Operatore Sanitario

2.1.2 Paziente Anziano

Il paziente anziano usa l'applicazione per comunicare poche informazioni essenziali. Non gestisce il proprio piano terapeutico, ma interagisce con quanto configurato dall'operatore: vede i farmaci, conferma l'assunzione, vede gli esercizi, li completa e registra l'umore del giorno.

Stakeholder	Goal	Subgoal	C	R	U	D
Paziente Anziano	Accesso	Login e registrazione associata a un operatore	X	X		
Paziente Anziano	Umore	Registra o aggiorna umore quotidiano	X	X	X	
Paziente Anziano	Esercizi fisici	Visualizza lista esercizi		X		
		Completa esercizio guidato	X	X		
Paziente Anziano	Farmaci	Visualizza farmaci per fascia oraria		X		
		Conferma o annulla assunzione	X	X	X	

Tabella 2: Goal e operazioni CRUD del Paziente Anziano

2.2 Diagramma di Contesto

Il diagramma di contesto rappresenta A.L.I.C.E. come una scatola nera posta al centro dell'interazione tra utenti e servizi esterni. L'operatore sanitario invia al sistema configurazioni e aggiornamenti relativi a pazienti, farmaci ed esercizi; in cambio riceve dashboard, metriche, storico dei log e suggerimenti sintetici. Il paziente invia conferme giornaliere, completamenti degli esercizi e valore dell'umore; in cambio riceve una home semplificata, l'elenco delle attività e le istruzioni operative.

Sul lato dei servizi, Supabase fornisce autenticazione, persistenza dei dati e controllo di visibilità tramite Row Level Security. La Groq API viene usata solo dalla route server-side dedicata al suggerimento AI: il browser non riceve la chiave del servizio esterno e il testo generato è destinato all'operatore.

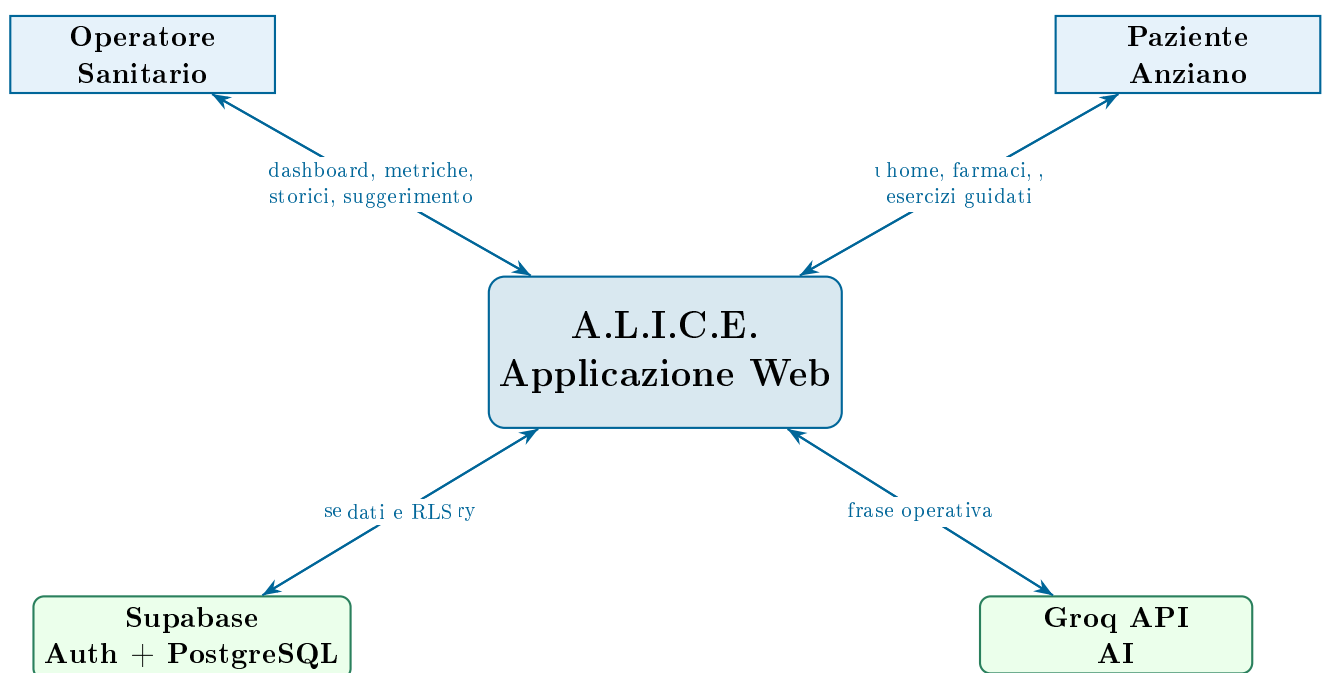


Figura 1: Diagramma di contesto del sistema A.L.I.C.E.

2.3 Scambio di Valore

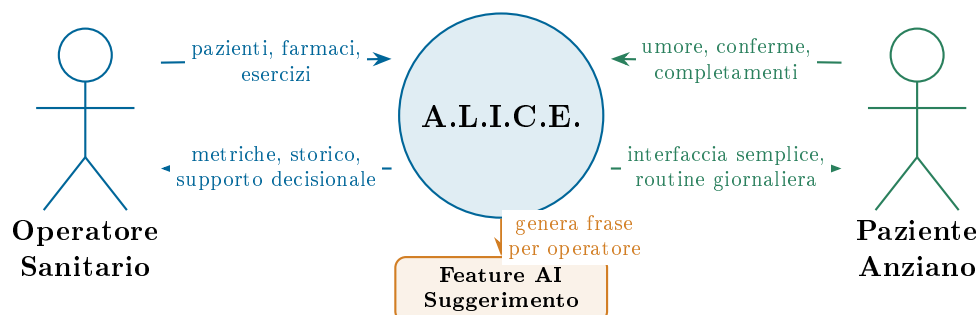


Figura 2: Scambio di valore con Operatore, Paziente e feature AI

3 Stato dell'Arte, Analisi della Concorrenza e Analisi “Make or Buy”

3.1 Stato dell'Arte

Il settore dell'assistenza domiciliare per anziani dispone oggi di numerose soluzioni digitali, ma spesso ciascuna copre un solo bisogno: promemoria farmacologici, teleassistenza, raccolta parametri o dashboard cliniche. A.L.I.C.E. si colloca in una zona intermedia: non vuole sostituire una cartella clinica professionale, ma integrare in una singola applicazione web le azioni quotidiane piu' rilevanti per il monitoraggio remoto.

Le tecnologie moderne rendono questo approccio sostenibile:

- **Framework full-stack:** Next.js permette di gestire pagine, rendering server-side, Server Components, Server Actions e route API nello stesso progetto.
- **Backend-as-a-Service:** Supabase riduce la complessita' del backend fornendo PostgreSQL, autenticazione, API auto-generate e Row Level Security.
- **Interfacce responsive:** l'applicazione e' utilizzabile da browser desktop e tablet, senza installazione.
- **AI come supporto operativo:** una route server-side puo' inviare dati sintetici a un modello esterno e ricevere suggerimenti testuali per l'operatore.

3.2 Analisi della Concorrenza

Sono stati considerati due competitor principali, gia' presenti nella versione finale del documento dello scorso appello.

3.2.1 MyTherapy

MyTherapy e' un'applicazione focalizzata sull'aderenza terapeutica. Gestisce promemoria per i farmaci, consente il tracciamento di parametri vitali e produce report utilizzabili dal personale medico. Il suo limite, rispetto ad A.L.I.C.E., e' la focalizzazione quasi esclusiva sulla terapia farmacologica: non offre una dashboard costruita attorno a esercizi fisici guidati e umore quotidiano con una vista anziano estremamente semplificata.

3.2.2 ADiLife

ADiLife e' un sistema professionale di teleassistenza per strutture sanitarie. Offre robustezza infrastrutturale e hardware dedicato, ma risulta piu' costoso e meno adatto a un progetto universitario basato su web app e strumenti open source. Il focus e' piu' orientato all'emergenza e al monitoraggio clinico professionale che alla routine quotidiana del paziente.

Piattaforma	Farmaci	Esercizi	Umore	Dashboard operatore
MyTherapy	✓	×	×	Parziale
ADiLife	✓	×	×	✓
A.L.I.C.E.	✓	✓	✓	✓

Tabella 3: Confronto sintetico tra A.L.I.C.E. e competitor

3.3 Analisi “Make or Buy”

Criterio	Make	Buy
Personalizzazione	Interfaccia progettata per due ruoli: operatore e paziente anziano.	Personalizzazione limitata ai vincoli del prodotto commerciale.
Costi	Bassi in fase iniziale grazie a Next.js, React e Supabase.	Licenze e costi di integrazione piu’ alti.
Integrazione	Farmaci, esercizi, umore e dashboard nello stesso modello dati.	Spesso richiede piu’ strumenti separati.
Controllo tecnico	Pieno controllo del codice, dello schema e delle regole RLS.	Dipendenza dal vendor.

Tabella 4: Analisi comparativa Make vs Buy

La scelta finale e’ **Make**: lo sviluppo interno consente di mantenere il progetto coerente con gli obiettivi didattici, con l’accessibilita’ richiesta e con la reale struttura software implementata.

4 Modello Informativo

Il modello informativo e' stato ricostruito a partire dallo schema Supabase presente nel file `supabase_reverse_engineered_schema.sql`. Il database contiene profili utente, specializzazioni per operatore e paziente, configurazioni operative e log giornalieri.

4.1 Diagramma EER

Il diagramma EER rappresenta le entita' principali e le relazioni tra esse, mantenendo uno stile vicino alla versione consegnata nel precedente appello. L'immagine PNG e' stata generata a partire dal sorgente Mermaid presente in `docs/diagrams/01_eer.mmd`.

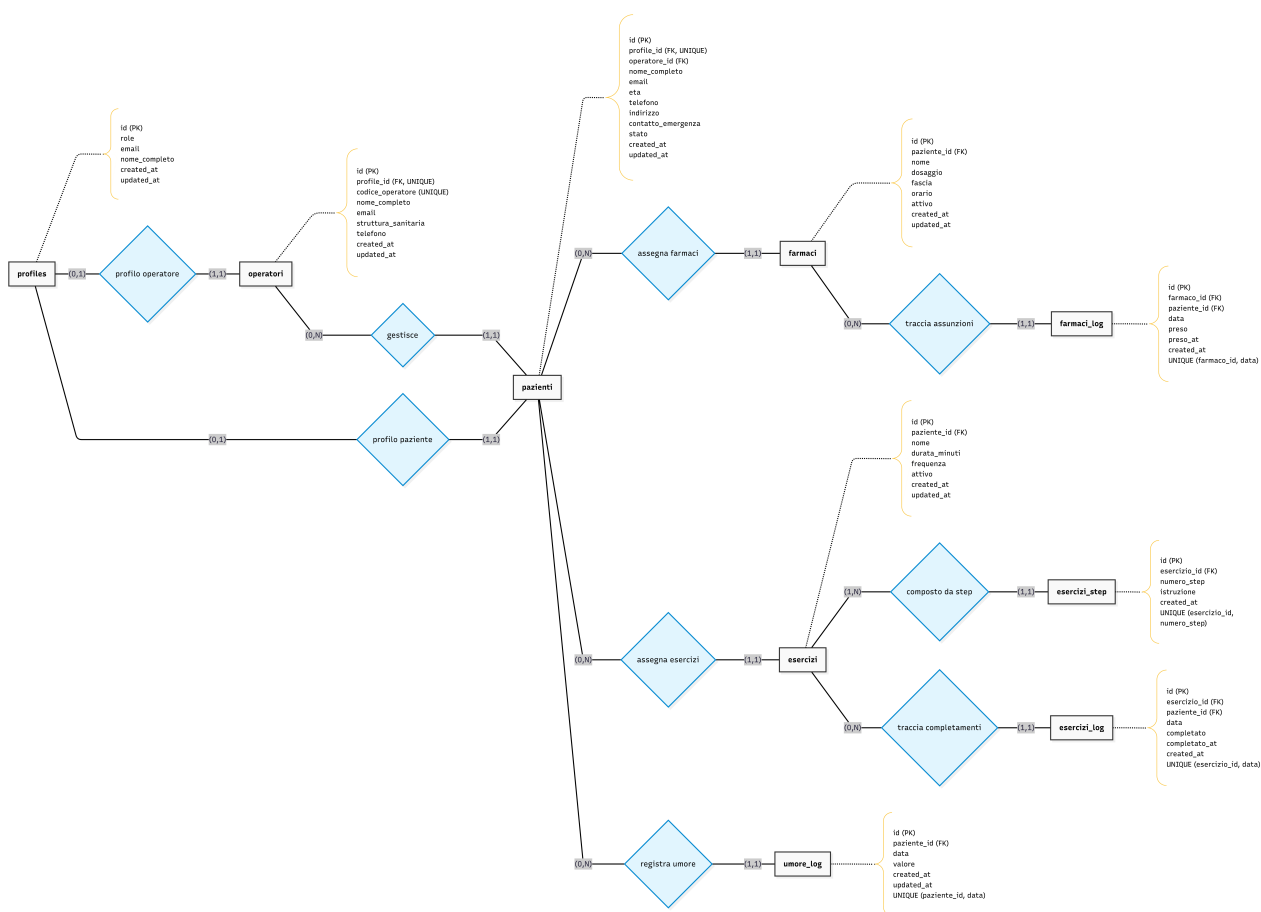


Figura 3: Diagramma EER aggiornato allo schema Supabase

Entita' principali

- **profiles:** profilo applicativo collegato a `auth.users`. Contiene ruolo, email e nome completo.
- **operatori:** dati dell'operatore sanitario, incluso codice operatore e struttura sanitaria.
- **pazienti:** dati del paziente, operatore assegnato, contatti e stato clinico.
- **farmaci:** farmaci assegnati al paziente, con dosaggio, orario, fascia e stato attivo.

- `farmaci_log`: registro giornaliero dell'assunzione dei farmaci.
- `esercizi`: esercizi fisici assegnati al paziente.
- `esercizi_step`: passaggi guidati che compongono un esercizio.
- `esercizi_log`: registro giornaliero degli esercizi completati.
- `umore_log`: registrazione giornaliera dell'umore del paziente.

4.2 Modello Relazionale

Il modello relazionale traduce il diagramma EER in tabelle, chiavi primarie, chiavi esterne e vincoli. Il diagramma seguente e' stato generato dal sorgente Mermaid docs/diagrams/02_modello_relazionale

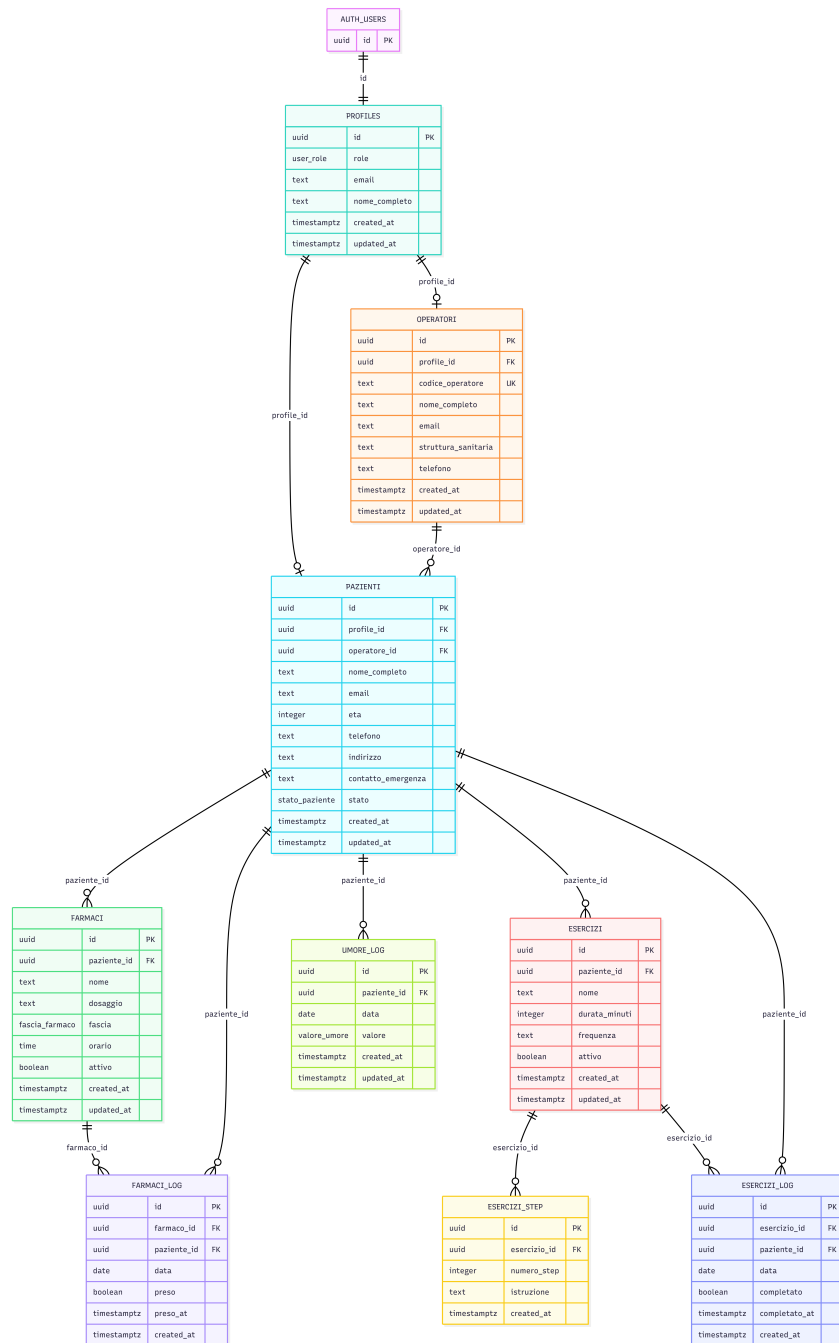


Figura 4: Diagramma del modello relazionale con tabelle, PK e FK

Tabella	Chiave primaria e campi principali	Relazioni e vincoli
profiles	id, role, email, nome_completo	id riferisce auth.users(id)

Tabella	Chiave primaria e campi principali	Relazioni e vincoli
operatori	id, profile_id, codice_operatore, nome_completo, struttura_sanitaria	profile_id e codice_operatore sono unici
pazienti	id, profile_id, operatore_id, eta, stato	profile_id unico; operatore_id riferisce operatori(id)
farmaci	id, paziente_id, nome, dosaggio, fascia, orario, attivo	paziente_id riferisce pazienti(id)
farmaci_log	id, farmaco_id, paziente_id, data, preso, preso_at	FK verso farmaci e pazienti; vincolo unico (farmaco_id, data)
esercizi	id, paziente_id, nome, durata_minuti, frequenza, attivo	paziente_id riferisce pazienti(id); durata_minuti > 0
esercizi_step	id, esercizio_id, numero_step, istruzione	FK verso esercizi; vincolo unico (esercizio_id, numero_step)
esercizi_log	id, esercizio_id, paziente_id, data, completato, completato_at	FK verso esercizi e pazienti; vincolo unico (esercizio_id, data)
umore_log	id, paziente_id, data, valore	FK verso pazienti; vincolo unico (paziente_id, data)

Tabella 5: Modello relazionale aggiornato allo schema Supabase

4.3 Stima Dimensionale del Database

4.3.1 Stima per entità

Tabella	Byte/riga stimati	Note
profiles	128	UUID, ruolo, email, nome e timestamp
operatori	384	Dati professionali, codice operatore e contatti
pazienti	512	Dati anagrafici, stato e contatto emergenza
farmaci	208	Nome, dosaggio, fascia, orario e FK
farmaci_log	96	FK, data, booleano e timestamp
esercizi	192	Nome, durata, frequenza e FK
esercizi_step	320	Numero step e istruzione testuale
esercizi_log	96	FK, data, booleano e timestamp
umore_log	80	Data e valore enum testuale

Tabella 6: Stima della dimensione media per riga

4.3.2 Tabella di popolamento e crescita

Tabella	Righe iniziali	Crescita annua	Byte/riga	Dimensione annua stimata
profiles	105	20	128	~ 16 KB
operatori	5	1	384	~ 2 KB
pazienti	100	20	512	~ 60 KB
farmaci	500	100	208	~ 122 KB
farmaci_log	4.500	182.500	96	~ 17,9 MB
esercizi	300	60	192	~ 67 KB
esercizi_step	900	180	320	~ 337 KB
esercizi_log	3.000	109.500	96	~ 10,8 MB
umore_log	3.000	36.500	80	~ 3,0 MB
Totale dopo un anno				~ 32,3 MB

Tabella 7: Stima del popolamento e della crescita annua del database

4.4 Modello di Visibilita'

Il modello di visibilita' e' implementato tramite Supabase Auth e policy Row Level Security. La logica principale e' semplice: l'operatore accede ai pazienti assegnati al proprio `operatore_id`; il paziente accede solo ai dati collegati al proprio profilo. I due diagrammi seguenti separano la visibilita' del paziente da quella dell'operatore.

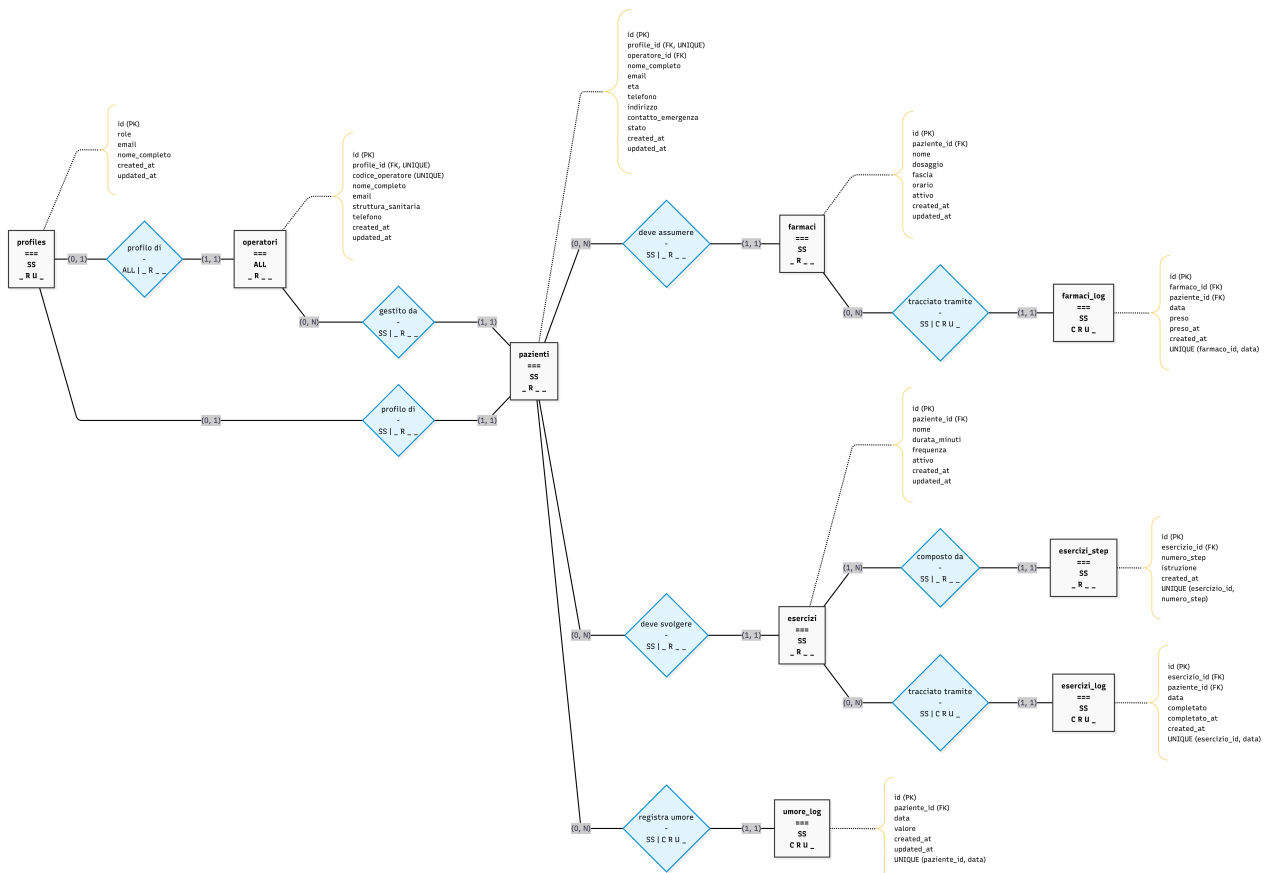


Figura 5: Modello di visibilita' RLS per il paziente

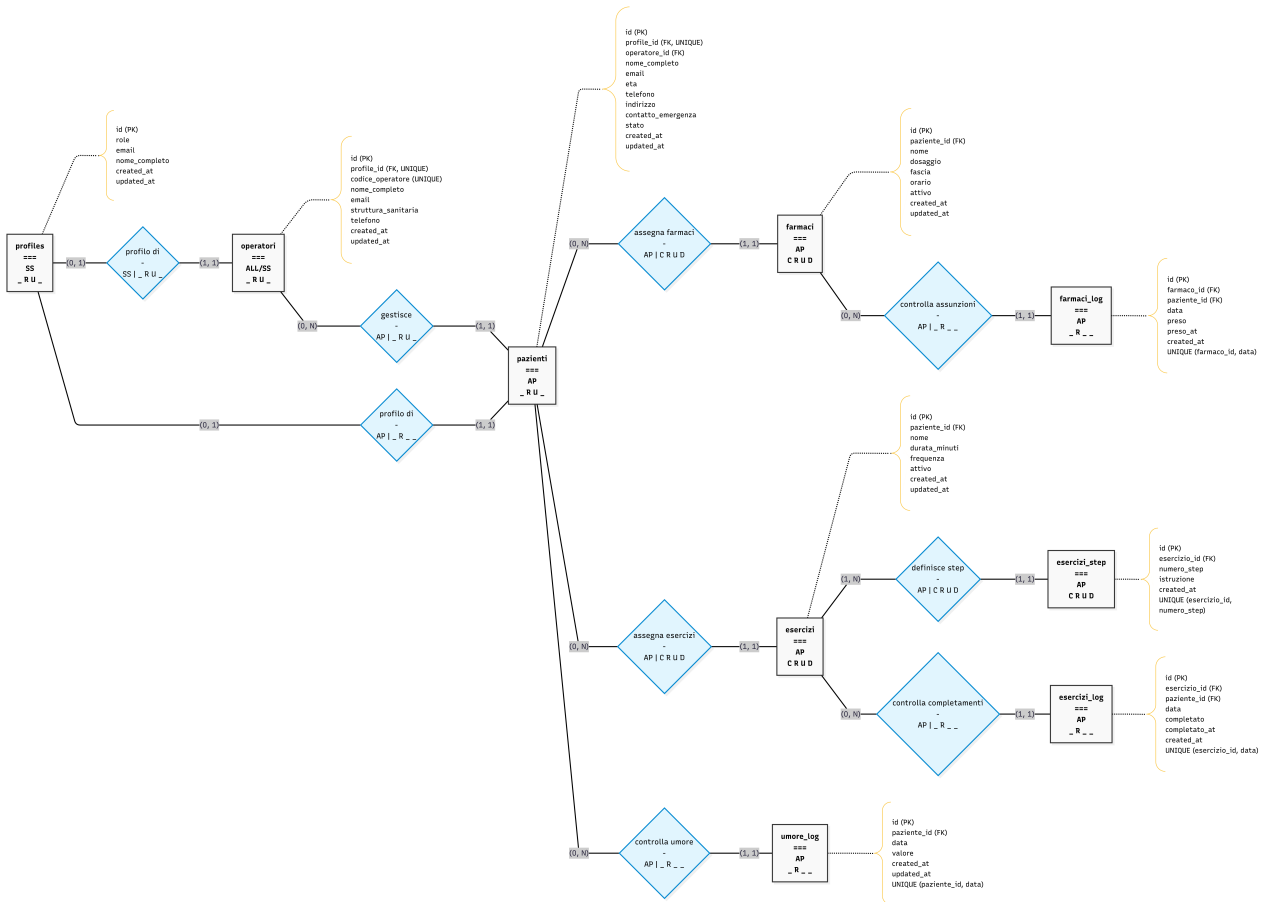


Figura 6: Modello di visibilità RLS per l'operatore

Risorsa	Operatore				Paziente			
	C	R	U	D	C	R	U	D
profiles		X	X			X	X	
operatori		X	X			X		
pazienti		X	X		X	X		
farmaci	X	X	X	X		X		
farmaci_log		X			X	X	X	
esercizi	X	X	X	X		X		
esercizi_step	X	X	X	X		X		
esercizi_log		X			X	X	X	
umore_log		X			X	X	X	

Tabella 8: Matrice CRUD di visibilità per ruolo

5 Modello di Navigazione

Il modello di navigazione e' stato ricavato dalla struttura reale di `src/app`. In Next.js App Router ogni cartella che contiene `page.tsx` diventa una pagina visitabile; per questo motivo il diagramma seguente non descrive funzionalita' ipotetiche, ma solo route presenti nel codice.

5.1 Legenda

Sigla	Significato
VP	Vista pubblica, accessibile senza autenticazione.
VF	Vista di form, usata per login o registrazione.
VD	Vista dashboard o vista di riepilogo.
VG	Vista gestionale, usata per modificare dati o configurazioni.
VA	Vista assistita per il paziente, con percorso semplice e pochi pulsanti.
API	Route non navigabile come pagina, usata dal codice applicativo.

Tabella 9: Legenda del modello di navigazione

5.2 Albero di navigazione reale

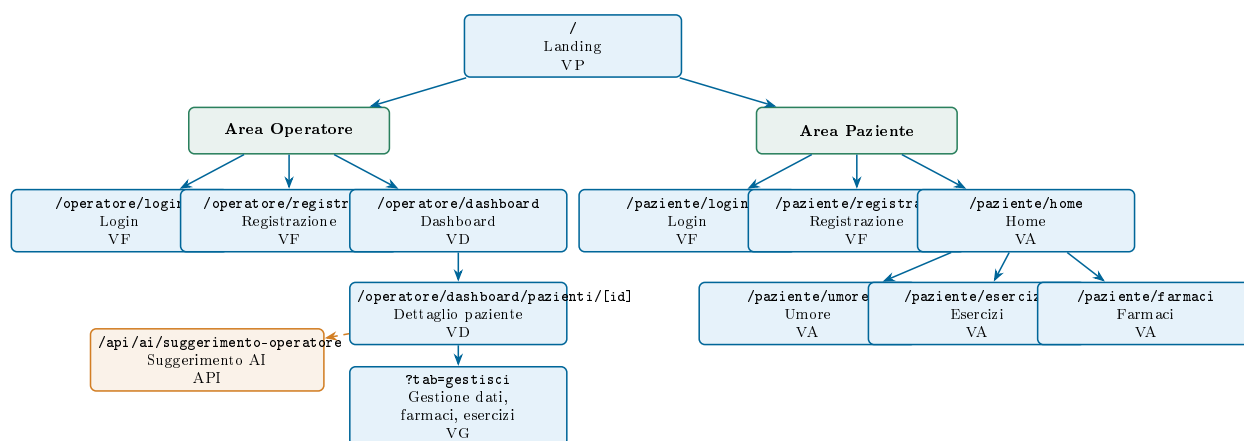


Figura 7: Albero di navigazione aggiornato alla struttura reale delle pagine

5.3 Descrizione delle viste

5.3.1 Area comune

- `/`: pagina iniziale. Presenta l'accesso alle due aree del sistema. Se l'utente e' gia' autenticato, il file `proxy.ts` lo reindirizza alla home corretta in base al ruolo.

5.3.2 Area operatore

- `/operatore/login`: form di accesso per l'operatore sanitario.
- `/operatore/registrati`: form di registrazione dell'operatore.
- `/operatore/dashboard`: vista principale dell'operatore con elenco pazienti, ricerca, filtri e metriche sintetiche.
- `/operatore/dashboard/pazienti/[id]`: dettaglio del paziente con dati anagrafici, farmaci, esercizi, umore, storico settimanale e suggerimento AI.
- `?tab=gestisci`: stato della pagina di dettaglio, non nuova route. Serve a modificare dati, farmaci ed esercizi.

5.3.3 Area paziente

- `/paziente/login`: form di accesso del paziente.
- `/paziente/registrati`: form di registrazione del paziente.
- `/paziente/home`: home semplificata con tre scelte principali.
- `/paziente/umore`: registrazione o aggiornamento dell'umore giornaliero.
- `/paziente/esercizi`: elenco degli esercizi assegnati e percorso guidato step-by-step.
- `/paziente/farmaci`: elenco dei farmaci del giorno, divisi per fascia oraria, con conferma di assunzione.

5.4 Regole di protezione e reindirizzamento

La navigazione privata viene controllata dal file `src/proxy.ts`. Il proxy legge la sessione Supabase e applica poche regole:

- utenti non autenticati verso `/operatore/dashboard` vengono mandati a `/operatore/login`;
- utenti non autenticati verso pagine private `/paziente/` vengono mandati a `/paziente/login`;
- utenti autenticati sulla landing vengono mandati alla home del proprio ruolo;
- utenti autenticati sulle pagine pubbliche di login o registrazione vengono reindirizzati alla propria area.

5.5 Principi di navigazione

1. **Separazione per ruolo**: operatore e paziente hanno percorsi distinti.
2. **Percorso breve per il paziente**: dalla home paziente si arriva a umore, esercizi e farmaci con un solo click.

3. **Dashboard unica per l'operatore:** l'operatore parte dalla dashboard e poi entra nel dettaglio del paziente.
4. **Nessuna pagina non implementata:** il modello contiene solo route realmente presenti nel codice.

6 Modello Architettuale

A.L.I.C.E. e' una web application full-stack basata su **Next.js App Router** e **Supabase**. Il progetto non usa un application server separato: Next.js gestisce pagine, componenti server, componenti client, Server Actions, route API e proxy; Supabase gestisce autenticazione, database PostgreSQL, API PostgREST e Row Level Security.

6.1 Architettura hardware

L'architettura hardware e' volutamente leggera. Gli utenti accedono tramite browser, mentre il codice applicativo viene eseguito su un ambiente serverless compatibile con Next.js. I dati persistenti e le sessioni sono gestiti da Supabase; il suggerimento AI viene generato tramite Groq API.

Componente	Servizio	Responsabilita'
Client	Browser web	Visualizzazione interfaccia e invio azioni utente.
Web app	Next.js serverless	Rendering, routing, Server Actions, proxy e route API.
Backend gestito	Supabase	Auth, PostgreSQL, PostgREST e policy RLS.
Servizio AI	Groq API	Generazione del suggerimento testuale per l'operatore.

Tabella 10: Componenti hardware e servizi esterni

6.2 Modello architetturale

Il modello architetturale non e' un MVC classico. La separazione delle responsabilita' e' ottenuta tramite il modello di Next.js App Router:

- le **pagine** definiscono le viste raggiungibili dall'utente;
- i **Server Components** caricano dati e preparano la UI lato server;
- i **Client Components** gestiscono interazioni complesse nel browser;
- le **Server Actions** gestiscono mutazioni come login, registrazione, farmaci ed esercizi;
- il **proxy** gestisce sessione e redirect delle aree private;
- Supabase rappresenta il livello dati e applica le policy di visibilita'.

Questa architettura riduce il numero di file necessari rispetto a un backend separato e mantiene la logica vicino alle pagine che la utilizzano.

6.3 Architettura software

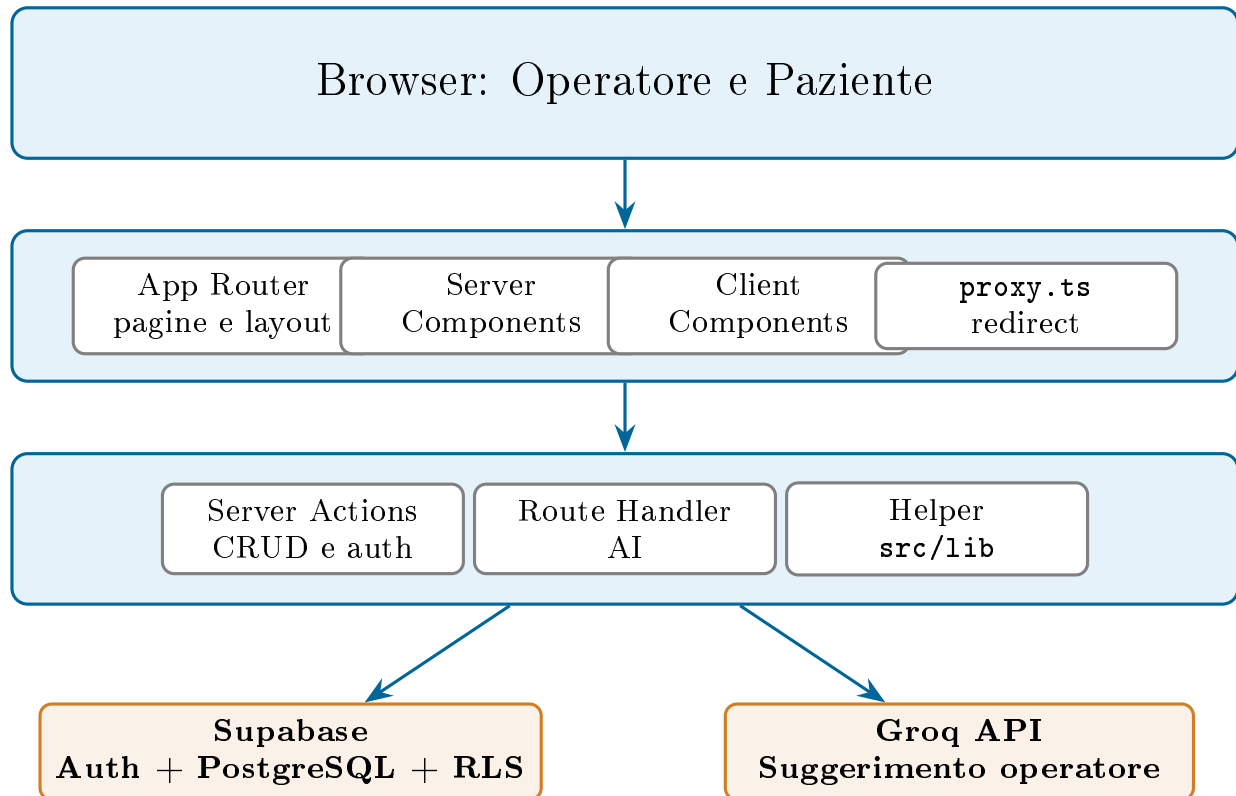


Figura 8: Architettura software a livelli del sistema A.L.I.C.E.

6.4 Responsabilit  dei livelli

Livello	Responsabilit�
Presentazione	Componenti React, form, dashboard, card e pagine semplificate del paziente.
Navigazione	App Router per le route e <code>proxy.ts</code> per redirect e sessione.
Logica applicativa	Server Actions, pagine server e helper in <code>src/lib</code> .
Dati	Tabelle Supabase, funzioni SQL, trigger e policy RLS.
Integrazione AI	Route <code>/api/ai/suggerimento-operatore</code> , chiamata server-side a Groq.

Tabella 11: Responsabilit  dei livelli software

6.5 Stack tecnologico

Tecnologia	Versione	Uso nel progetto
<code>next</code>	16.1.6	Framework full-stack, App Router, build e proxy.
<code>react</code> / <code>react-dom</code>	19.2.3	Componenti dell'interfaccia utente.
<code>@supabase/ssr</code>	0.8.0	Client Supabase compatibile con server, client e proxy.
<code>typescript</code>	5.x	Tipizzazione del codice applicativo.
CSS Modules / CSS globale	n.d.	Stile delle pagine senza librerie UI esterne.
Groq API	via <code>fetch</code>	Suggerimento AI lato server.

Tabella 12: Stack tecnologico realmente presente nel progetto

6.6 DBMS, browser e sistemi supportati

Il DBMS è PostgreSQL tramite Supabase. Il codice applicativo usa il client Supabase per leggere e scrivere dati, mentre le regole RLS nel database impediscono l'accesso a dati non autorizzati. L'applicazione è fruibile da browser moderni su Windows, macOS, Linux, Android e iPadOS. Per il paziente è consigliato un tablet o un PC con schermo ampio; per l'operatore è sufficiente un PC, laptop o tablet.

7 Aspetti Implementativi

Gli aspetti implementativi descrivono come l'architettura viene tradotta in file e responsabilità concrete. Il progetto usa una struttura semplice, coerente con Next.js App Router: le pagine stanno in `src/app`, i componenti riutilizzabili in `src/components`, le funzioni di supporto in `src/lib` e la protezione della navigazione in `src/proxy.ts`.

7.1 Struttura del progetto

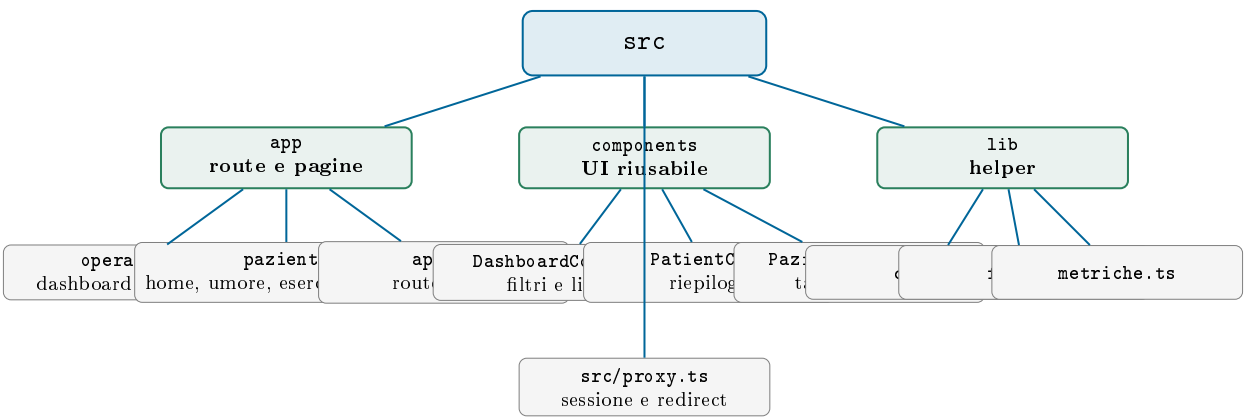


Figura 9: Struttura implementativa principale del progetto

7.2 Pagine e route

Le route principali sono separate per ruolo. L'area operatore contiene login, registrazione, dashboard e dettaglio paziente. L'area paziente contiene login, registrazione, home e le tre funzioni giornaliere: umore, esercizi e farmaci. La route `/api/ai/suggerimento-operatore` non è una pagina, ma un endpoint server-side richiamato dal dettaglio paziente.

File	Responsabilità
<code>src/app/operatore/dashboard/page.tsx</code>	Carica l'operatore, i pazienti assegnati e le metriche di riepilogo.
<code>src/app/operatore/dashboard/paziente/page.tsx</code>	Verifica che il paziente appartenga all'operatore e prepara dati di dettaglio.
<code>src/app/paziente/home/page.tsx</code>	Mostra la home semplificata del paziente.
<code>src/app/paziente/umore/page.tsx</code>	Salva o aggiorna l'umore giornaliero.
<code>src/app/paziente/esercizi/page.tsx</code>	Gestisce l'esecuzione guidata e il log degli esercizi.
<code>src/app/paziente/farmaci/page.tsx</code>	Mostra farmaci per fascia e registra l'assunzione.

Tabella 13: Pagine principali

7.3 Server Actions e accesso ai dati

Le Server Actions gestiscono le operazioni che modificano dati: registrazione, login, logout, aggiornamento del paziente, aggiunta o rimozione di farmaci ed esercizi. La scelta delle Server Actions evita controller REST manuali per ogni form e mantiene la logica lato server.

File	Operazioni principali
<code>src/app/operatore/actions.ts</code>	Login, registrazione e logout operatore.
<code>src/app/paziente/actions.ts</code>	Login, registrazione e logout paziente.
<code>src/app/operatore/dashboard/pazienti/actions.ts</code>	create/delete farmaco, create/delete esercizio.

Tabella 14: Server Actions del progetto

7.4 Componenti client

I componenti client sono usati dove serve interazione nel browser. `DashboardContent.tsx` gestisce ricerca e filtri nella dashboard; `PatientCard.tsx` visualizza un paziente in forma sintetica; `PazienteDettaglio.tsx` gestisce tab, modali, form, storico e richiesta del suggerimento AI.

7.5 Helper di supporto

Durante il refactoring alcune funzioni ripetute sono state spostate in `src/lib.date.ts` gestisce la data corrente, `form.ts` centralizza la lettura dei valori da `FormData`, `metriche.ts` calcola percentuali e `paziente-client.ts` recupera l'id del paziente associato all'utente autenticato.

7.6 Suggerimento AI

Il suggerimento AI e' isolato nella route `src/app/api/ai/suggerimento-operatore/route.ts`. Il client invia dati sintetici e non identificativi; la route usa la chiave `GROQ_API_KEY` lato server, chiama Groq tramite `fetch` e restituisce una sola frase per l'operatore. Questa separazione evita di inserire la chiave nel browser e mantiene la logica AI fuori dal componente React.

7.7 Sicurezza

La sicurezza si basa su Supabase Auth, proxy Next.js e Row Level Security. Il proxy protegge le pagine private e gestisce i redirect; le pagine server controllano che l'operatore acceda solo ai propri pazienti; le policy RLS nel database impediscono letture o modifiche non coerenti con il ruolo.

8 Test

I test del progetto sono stati organizzati in modo pratico, controllando sia la correttezza tecnica della build sia i flussi principali usati da operatore e paziente.

8.1 Strumenti utilizzati

- **Build Next.js:** il comando `npm.cmd run build` controlla compilazione, tipi TypeScript e generazione delle route.
- **Browser e DevTools:** usati per verificare navigazione, responsive design, richieste di rete e assenza di errori runtime.
- **Supabase Dashboard:** usata per controllare dati inseriti, tabelle, funzioni SQL e policy RLS.
- **Postman:** usato per testare manualmente le API Supabase e, quando configurata la chiave, la route del suggerimento AI.

8.2 Casi di test principali

Area	Test	Risultato atteso
Autenticazione	Login e registrazione separati per operatore e paziente.	L'utente entra solo nella propria area.
Proxy	Accesso diretto a pagine private senza sessione.	Redirect alla pagina di login corretta.
Dashboard	Ricerca e filtro dei pazienti.	La lista mostra solo i pazienti corrispondenti.
Dettaglio paziente	Apertura di un paziente associato all'operatore.	Vengono mostrati dati, farmaci, esercizi, umore e storico.
Farmaci	Conferma giornaliera di un farmaco lato paziente.	Viene creato o aggiornato il record in <code>farmaci_log</code> .
Esercizi	Completamento di un esercizio guidato.	Viene salvato il completamento in <code>esercizi_log</code> .
Umore	Inserimento o modifica dell'umore del giorno.	Viene salvato un solo valore per paziente e data.
AI	Richiesta di suggerimento dal dettaglio paziente.	La route restituisce una frase breve per l'operatore.

Tabella 15: Casi di test funzionali

8.3 Verifiche sul database

Le verifiche su Supabase hanno controllato:

- creazione automatica del profilo dopo la registrazione;
- associazione corretta tra operatore e paziente;
- inserimento dei log giornalieri per farmaci, esercizi e umore;
- visibilita' dei dati coerente con le policy RLS.

9 Conclusioni

Il progetto A.L.I.C.E. realizza una piattaforma web semplice per il monitoraggio remoto di pazienti anziani. La soluzione mette insieme tre aree concrete: farmaci, esercizi fisici e umore quotidiano. L'operatore usa una dashboard per controllare i pazienti, mentre il paziente usa pagine lineari con azioni brevi e leggibili.

9.1 Risultati raggiunti

- **Separazione dei ruoli:** operatore e paziente hanno pagine, login e percorsi distinti.
- **Dashboard operatore:** l'operatore visualizza pazienti, stato, metriche e dettaglio individuale.
- **Area paziente semplificata:** il paziente accede a umore, esercizi e farmaci partendo da una home con tre scelte principali.
- **Database coerente:** lo schema contiene tabelle dedicate a profili, operatori, pazienti, farmaci, esercizi e log.
- **Navigazione protetta:** il proxy Next.js gestisce i redirect principali in base alla sessione Supabase.
- **Suggerimento AI:** l'operatore puo' ricevere una frase sintetica di supporto basata sui dati del paziente.

9.2 Benefici attesi

Per il paziente

- interfaccia piu' semplice rispetto a un gestionale sanitario tradizionale;
- meno passaggi per registrare umore, farmaci ed esercizi;
- conferma immediata delle azioni completate.

Per l'operatore

- visione centralizzata dei pazienti seguiti;
- controllo piu' rapido di aderenza farmacologica, esercizi e umore;
- possibilita' di modificare piano e dati del paziente da una sola pagina.

9.3 Limiti attuali

Il progetto copre il flusso principale, ma resta una piattaforma didattica. Alcune funzioni sono volutamente semplici: non sono presenti notifiche automatiche, videochiamate, app mobile nativa o integrazione con dispositivi medici. Questa scelta mantiene il codice piu' leggibile e adatto alla presentazione universitaria.

9.4 Sviluppi futuri

Possibili miglioramenti futuri sono:

- notifiche per ricordare al paziente farmaci ed esercizi;
- grafici piu' avanzati per lo storico dell'operatore;
- esportazione dei report del paziente;
- videochiamata semplificata tra operatore e paziente;
- app mobile o Progressive Web App installabile.

10 Matrice RACI

La matrice RACI chiarisce la distribuzione delle responsabilità tra i membri del gruppo. Le sigle utilizzate sono:

- **R** – Responsible: esegue l'attività;
- **A** – Accountable: approva il risultato;
- **C** – Consulted: fornisce supporto o revisione;
- **I** – Informed: viene informato sull'esito.

10.1 Ruoli del team

Studente	Responsabilità prevalente
Manuel Manna	Coordinamento della relazione, contesto, stato dell'arte, revisione finale.
Alessio Mininni	Implementazione, refactoring, Supabase, architettura software e allineamento codice-documentazione.
Andrius Tamuliunaite	Analisi dei modelli, verifica funzionale, supporto alla navigazione e revisione dei contenuti.

Tabella 16: Ruoli del team

10.2 RACI della documentazione

Attività	Manna	Mininni	Tamuliunaite
Capitolo 1: contesto e obiettivi	A/R	C	C
Capitolo 2: stakeholder, goal e valore	C	C	A/R
Capitolo 3: stato dell'arte e make or buy	A/R	I	C
Capitolo 4: EER, relazionale e visibilità	C	A/R	C
Capitolo 5: modello di navigazione	C	A/R	C
Capitolo 6: modello architetturale	C	A/R	I
Capitolo 7: aspetti implementativi	I	A/R	C
Capitoli 8-9: test e conclusioni	A	C	R
Revisione finale della relazione	A/R	C	C

Tabella 17: Matrice RACI della documentazione

10.3 RACI dello sviluppo

Attività'	Manna	Mininni	Tamuliunaite
Configurazione progetto Next.js	I	A/R	C
Schema Supabase e policy RLS	C	A/R	C
Autenticazione operatore e paziente	I	A/R	C
Dashboard operatore	C	A/R	C
Dettaglio paziente e gestione farmaci/esercizi	C	A/R	C
Pagine paziente: home, umore, esercizi, farmaci	C	A/R	R
Proxy Next.js e protezione navigazione	I	A/R	C
Route AI con Groq	I	A/R	C
Refactoring helper in <code>src/lib</code>	I	A/R	C
Test manuali dei flussi principali	C	C	A/R

Tabella 18: Matrice RACI dello sviluppo